

claude-windows / 45ea3a91-654d-4972-9cd3-65f35db54caf

Metadata

```
- Source: `claude-windows`  
- Kind: `claude`  
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\45ea3a91-654d-4972-9cd3-65f35db54caf\subagents\workflows\wf_2a64bc88-2a8\agent-a867f2a9e52add6dc.jsonl`  
- SHA-256: `f408461e57a97bcd6f74f061f1a0bbea82d3749ed1a86958f7eef3cd67698619`  
- Source modified: `2026-06-10T06:10:55+00:00`  
- Imported at: `2026-07-05T16:48:27+00:00`  
- project: `wf_2a64bc88-2a8`  
- session_id: `45ea3a91-654d-4972-9cd3-65f35db54caf`
```

Transcript

1. user (2026-06-10T06:09:07.939Z)

CONTEXT ? two sibling repos on a Windows machine:

- INDEXER: C:/Users/avidu/Projects/badminton-highlight-indexer ? local AI badminton(->racket-sports) highlight indexer + rally detector. FastAPI + SQLite + ffmpeg + GPU CV via WSL; Gemini = paid cloud AI. docs/ tree is rich. Current branch docs/next-steps-multisport (master = main branch).

- COLLECTOR: C:/Users/avidu/Projects/sports-data-collector ? golden-label acquisition/curation/ingestion CLI (system-of-record candidate for training corpus).

Project state (June 2026): scaffolding complete; owned-model roadmap agreed (docs/OWNED_MODEL_IMPLEMENTATION_PLAN.md) but NO platform/owned-model implementation started; next committed platform slice = async job model. Licensing guardrail: MIT/Apache-2.0/BSD only for code+data+weights; paid LLMs are inference-only, never training-data sources. ENVIRONMENT RULES: this dev environment has NO GPU/torch/cv2 ? do NOT attempt to run the video pipeline or import torch/cv2 modules. Test suites are offline/fully-mocked and SAFE to run. You are READ-ONLY: never edit/create/delete repo files; running tests and git read commands is allowed.

QUALITY BAR: cite file paths (and line numbers where possible) for every finding. Report what IS in the code/docs, not what you assume. Be skeptical and concrete. Your final structured output is consumed programmatically by an orchestrator ? return raw data, not prose for a human.

ROLE: adversarial verifier. An auditor ("docs-strategy") claimed the finding below. Try to REFUTE it against the actual files. Default to isReal=false if the evidence does not hold up, the behavior is documented/deliberate (e.g. listed in docs/CODE_MAP.md gotchas or DEFERRED_HARDENING_PLAN.md as a known deferral ? unless the auditor adds genuinely NEW consequence), or the severity is materially overstated.

FINDING: {"title":"README claims default segmenter is yolo_hybrid; the checked-in config.json ships 'gemini' (paid path), and the file wins at runtime","severity":"high","area":"README vs config reality","file":"C:/Users/avidu/Projects/badminton-highlight-indexer/config.json:26","evidence":"README.md:74: 'using whichever segmenter is set as `default\_segmenter` in `config.json` (`yolo\_hybrid` by default)'. Reality: config.json:26 ships `\"default\_segmenter\": \"gemini\"`; backend/config/models.py:47 has the code default 'yolo_hybrid'; docs/CODE_MAP.md:102 confirms the precedence: '? `indexing.default\_segmenter=\\'gemini\\'` diverges from model default `\\'yolo\_hybrid\\'` ? file wins at runtime' (also CODE_MAP.md:238).","why_it_matters":"A fresh user following README's CLI quickstart would unknowingly route their video to the paid Gemini cloud path (or hit a silent provider no-op without an API key ? a failure mode the observability spec itself names 'silent_provider_noop'), instead of the local segmenter README promises. CODE_MAP documents the divergence but README asserts the wrong default as fact."}

Steps: open the cited file(s) and any counter-evidence files; verify quotes/line claims; judge severity honestly. Cheap greps allowed; do NOT run the pipeline. Return isReal, confidence, and a 1-3 sentence verdict (with corrected detail if partially right).

2. user (2026-06-10T06:09:17.446Z)

55 > The default `yolo\_hybrid` segmenter uses **torchvision**'s COCO-pretrained Faster

R-CNN, whose weights are downloaded and cached automatically by torchvision on first run ?
nothing is bundled in the repo. (Pick a lighter/heavier detector via `indexing.detector\_model`:
`fasterrcnn\_resnet50\_fpn` (default), `fasterrcnn\_mobilenet\_v3\_large\_fpn`, or
`retinanet\_resnet50\_fpn`.)

```
56
57     ---
58
59     ## ? Running the Application
60
61     ### 1. Launching the Local Web App Dashboard
62     To start the FastAPI server and open the web dashboard:
63     ```bash
64     python -m backend.main --server --port 8000
65     ```
66     Open your browser and navigate to [http://localhost:8000](http://localhost:8000).
67     *Simply input your local raw video file path in the sidebar to process, validate, index,
68     and compile highlights instantly.*
69
70     ### 2. Running in CLI Processing Mode
71     To process and index a video completely via terminal commands:
72     ```bash
73     python -m backend.main --video-path "C:/path/to/my_video.mp4"
74     ```
75     This outputs a validation summary and indexes all rallies directly into the local SQLite
76     database, using whichever segmenter is set as `default_segmenter` in `config.json`
77     (`yolo_hybrid` by default).
78
79     To explicitly pick a segmenter for one run:
80     ```bash
81     python -m backend.main --video-path "C:/path/to/my_video.mp4" --segmenter yolo_hybrid
82     python -m backend.main --video-path "C:/path/to/my_video.mp4" --segmenter gemini
83     python -m backend.main --video-path "C:/path/to/my_video.mp4" --segmenter motion
84     ```
85
86     ### 3. Rich CLI Debugging (`--debug-clip`)
87     To diagnose exactly why a video segment was registered as active/dead play, or inspect
88     detailed focus/continuity metrics at a particular time in seconds:
89     ```bash
90     python -m backend.main --video-path "C:/path/to/my_video.mp4" --debug-clip 125.5
91     ```
92     *Outputs granular validation check logs and motion ratios within a +/- 3 second frame
93     window around the timestamp.*
94
95     ### 4. Extracting an Arbitrary Trimmed Segment (`--trim-start` / `--trim-end`)
96     For fast iteration on a tiny slice of a multi-GB recording, you can stream-copy a
97     segment to disk without re-encoding:
98     ```bash
99     python -m backend.main --video-path "C:/path/to/my_video.mp4" \
100         --trim-start 600 --trim-end 660 --trim-output slice_10m_to_11m.mp4
```

3. user (2026-06-10T06:09:18.188Z)

```
1      {
2          "sport": "badminton",
3          "ai_provider": "gemini",
4          "ai_model": "gemini-2.5-flash",
5          "validation": {
6              "min_resolution_width": 1280,
7              "min_resolution_height": 720,
8              "min_fps": 29.9,
9              "min_blur_threshold": 20.0,
10             "max_scene_cuts_per_minute": 0.5,
11             "relevance": {
12                 "court_green_lower": [
13                     35,
```

```

14         40,
15         40
16     ],
17     "court_green_upper": [
18         85,
19         255,
20         255
21     ],
22     "court_pixels_min_ratio": 0.05
23 }
24 },
25 "indexing": {
26     "default_segmenter": "gemini",
27     "use_gpu": true,
28     "motion_threshold": 0.08,
29     "min_segment_duration": 3.0,
30     "dead_time_merge_gap": 2.0,
31     "chunk_duration": 90,
32     "chunk_overlap": 10,
33     "detector_model": "fasterrcnn_resnet50_fpn",
34     "detector_score_threshold": 0.5,
35     "yolo_velocity_threshold": 0.15,
36     "yolo_frame_skip": 3,
37     "yolo_tracker": "bytetrack.yaml",
38     "yolo_parallel_chunks": false,
39     "yolo_chunk_workers": 2,
40     "min_action_window_duration": 2.0,
41     "log_yolo_candidates": true,
42     "store_yolo_candidates": true,
43     "ai_handoff_padding": 2.0,
44     "ai_max_workers": 5,
45     "tracknet": {
46         "wsl_distro": "Ubuntu",
47         "repo_dir": "~/models/TrackNetV4",
48         "conda_sh": "~/miniconda3/etc/profile.d/conda.sh",
49         "conda_env": "TrackNetV4",
50         "weights_path": "",
51         "queue_length": 5,
52         "stage_in_wsl": true,
53         "wsl_stage_dir": "~/clips",
54         "timeout_sec": 0,
55         "velocity_threshold": 0.02
56     }
57 },
58 "output": {
59     "default_highlights_name": "highlights.mp4",
60     "db_name": "sports_indexer.db"
61 },
62 "stitching": {
63     "begin_padding": 0.5,
64     "end_padding": 0.5,
65     "use_cache": false,
66     "min_shot_count": 1
67 }
68 }

```

4. user (2026-06-10T06:09:25.129Z)

```

35     repo_dir: str = "~/models/TrackNetV4"
36     conda_sh: str = "~/miniconda3/etc/profile.d/conda.sh"
37     conda_env: str = "TrackNetV4"
38     weights_path: str = ""
39     queue_length: int = 5
40     stage_in_wsl: bool = True
41     wsl_stage_dir: str = "~/clips"

```

```

42         timeout_sec: int = 0
43         velocity_threshold: float = 0.02
44
45
46     class IndexingConfig(BaseModel):
47         default_segmenter: str = "yolo_hybrid"
48         use_gpu: bool = True
49         motion_threshold: float = 0.08
50         min_segment_duration: float = 3.0
51         dead_time_merge_gap: float = 2.0
52         chunk_duration: float = 90.0
53         chunk_overlap: float = 10.0
54         detector_model: str = "fasterrcnn_resnet50_fpn"
55         detector_score_threshold: float = 0.5
56         yolo_velocity_threshold: float = 0.15
57         yolo_frame_skip: int = 3
58         yolo_tracker: str = "bytetrack.yaml"
59         yolo_parallel_chunks: bool = False
60         yolo_chunk_workers: int = 2
61         min_action_window_duration: float = 2.0
62         log_yolo_candidates: bool = True
63         store_yolo_candidates: bool = True
64         ai_handoff_padding: float = 2.0

```

5. user (2026-06-10T06:09:26.182Z)

config.json
d19557e feat(indexer): replace AGPL ultralytics YOLO with permissive torchvision + ByteTrack

6. user (2026-06-10T06:09:27.416Z)

```

27-         [validation FAIL -> persist status="failed", return early; report STILL emitted in
finally]
28- -> db.add_video(video_id, filepath, status, [r.dict()])
@backend/infrastructure/database.py
29: -> seg_name = req.segmenter or global_config.indexing.default_segmenter (fallback 'motion')
30- -> seg = segmenter_registry.get(seg_name)(db, global_config) ?
@backend\pipeline\segmenters\base.py::segmenter_registry (400 + available list if missing)
31- -> segments = seg.process_video(video_id, video_path, player_pool?, max_frames?)
--
84- - `::load_config` ? ? legacy thin wrapper returning a **dict**
(`load_app_config(path).model_dump(...)`); kept for transition ? new code imports
`load_app_config`/`AppConfig` directly. `::ProcessRequest`/`::QueryRequest`/`::CompileRequest`
pydantic bodies.
85- - `main()` sets `global_config.output.output_dir = args.output_dir` (typed field on
`OutputConfig`); `db_path = os.path.join(global_config.output.output_dir, output.db_name)`.
`compile_highlights` reads the SAME `global_config.output.output_dir`, so `--output-dir` is
honored end-to-end (the old write-only `runtime_overrides` hack is gone).
86: - ? `--segmenter` argparse `choices` frozen at build time from
`segmenter_registry.list_available()` ? a lazily-registered segmenter wouldn't appear. CLI
default_segmenter fallback `motion`.
87-- `@run_process_and_stitch.py` ? one-shot demo; ? hardcoded paths, interactive `input()`
(blocks automation), skips validation. Config via `backend.config.load_config` (typed);
segmenter, padding, and db path all read from the model (no literal fallbacks).
88-- `@run_eval.py::main` ? score one video's DB preds vs GT CSV.
`--stage{candidates,final,both}`. Exit 2=arg/IO. ? pure scorer; errors if video never processed.
`::default_db_path` resolves via `backend.config.load_config`
(`output.output_dir`/`output.db_name`). `get_file_md5` is an alias for `compute_video_id`.
--
94- - `::AppConfig` ? root. Scalars `sport='badminton'`, `ai_provider='gemini'`,
`ai_model='gemini-2.5-flash'`; sections `validation`, `indexing`, `output`, `stitching`
(`default_factory`). `model_config = {extra:'allow', validate_assignment:True}`. ?
`extra='allow'` -> unknown/typo'd config.json keys silently kept (legacy tolerance).
95- - ? `::AppConfig.get` ? legacy dict-compat shim: returns nested **sections as plain dicts**
(`model_dump(mode='json')`, NOT the model) so chained `config.get("indexing",{}).get(...)` keeps

```

```

working; falls back to searching a leaf key in any section (sections discovered dynamically from
`model_fields`). ? "bit every validator on real runs" per docstring ? returning the model breaks
`.get`. `::AppConfig.__getitem__` raises `KeyError` for unknown top-level keys else delegates to
`.get`.
96:[Omitted long matching line]
97:[Omitted long context line]
98-- `@backend/config/loader.py` ? load/save with defaults-merge.
--
100- - `::load_config` ? precedence built-in-defaults -> file overrides. ? **shallow top-level
merge** (`**defaults`, **file_data`): a section in config.json REPLACES the whole defaults dict
for that section (Pydantic then re-applies field defaults for missing leaves).
Missing/unreadable/parse-error file -> fully-defaulted AppConfig + `[CONFIG WARNING]` (non-
fatal).
101- - `::save_default_config` ? writes fresh `config.json`, **overwrites** if present (used by
setup.py/--setup). `::get_default_config` ? ? transitional plain-dict alias.
102:- `@config.json` ? on-disk config; mirrors `AppConfig`. ?
**`indexing.default_segmenter='gemini'`** diverges from model default `yolo_hybrid` ? file
wins at runtime.
103-- `@setup.py` ? `::init_default_config` ? delegates to `backend.config.save_default_config`
(single-source defaults; skips if config.json exists). `::run_pip_install` GPU-aware (`nvidia-
smi` -> cu124 else cpu). `::run_diagnostics` (Python?3.8, ffmpeg/ffprobe on PATH, optional
torch+CUDA).
104-
--
175-### 2.8 Reporting / observability ? (per-video, soft-dep `obsreport`)
176:[Omitted long context line]
177:[Omitted long matching line]
178:[Omitted long context line]
179-
--
236-- **MD5 keying:** `video_id` = MD5 of WHOLE file via the single
`@backend/utils/hashing.py::compute_video_id` helper (64KB chunks), imported by main + all
run_*.py. Manifest `video_id` is a label, NOT the DB key.
237-- **Config access:** `backend/main.py` and all four `run_*.py` load via
`backend.config.load_config` (typed `AppConfig`). The legacy `.get()` dict-compat shim remains
for validators/segmenters. Remaining literal: a defensive `getattr(..., 'motion')` fallback in
`main.py` that only fires if `global_config`/.indexing` is `None` (the ASGI-import edge case).
238:- **Segmenter default:** single source of truth = model
`IndexingConfig.default_segmenter='yolo_hybrid'`; `config.json` may override at runtime (ships
`gemini`). run_*.py read the model value (no literals); `main.py` keeps the defensive `motion`
getattr fallback noted above.
239-- **`AppConfig.get` must return dict sections, not models:** the legacy
`.get("indexing",{}).get(...)` chains everywhere depend on it; the `extra="allow"` model also
silently keeps typo'd config keys (no rejection).
240-- **`output_dir`:** a real field on `OutputConfig` (default `output`); the CLI `--output-
dir` overrides it on the typed model in `main()`, and `compile_highlights` reads the same
`global_config.output.output_dir` ? honored end-to-end.
--
262-| Task | Open |
263-|---|---|
264-| Change a config default / add a config field | `@backend/config/models.py` (single source
of truth) -> regenerate via `setup.py`/`save_default_config`; ? config.json may override
(default_segmenter) |
265-| Read config in new code | `from backend.config import load_config, AppConfig`
(`@backend/config/__init__.py`); attribute access, NOT raw `json.load` |
266-| Tune candidate windowing (velocity\merge\min-dur) |
`@backend\pipeline\detectors\tracknet_runner.py::trajectory_to_action_windows`; cfg `config.json
indexing.{wasb,tracknet}.velocity_threshold`, `dead_time_merge_gap`,
`min_action_window_duration` |

```

7. user (2026-06-10T06:09:47.168Z)

```

1      """Configuration loader and saver.
2
3      Provides a single place to load validated AppConfig with built-in defaults

```

```

4     merged with optional file overrides (config.json).
5
6     This replaces scattered json.load + .get() patterns throughout the codebase.
7     """
8
9     from __future__ import annotations
10
11     import json
12     import logging
13     from pathlib import Path
14     from typing import Any
15
16     from .models import AppConfig
17
18     logger = logging.getLogger(__name__)
19
20     DEFAULT_CONFIG_PATH = Path("config.json")
21
22
23     def _get_builtin_defaults() -> dict[str, Any]:
24         """Return a plain dict of the current built-in defaults.
25
26         The AppConfig model is the single source of truth for defaults.
27         """
28         return AppConfig().model_dump(mode="json")
29
30
31     def load_config(path: str | Path | None = None) -> AppConfig:
32         """Load configuration with the following precedence:
33
34         1. Built-in defaults defined in the Pydantic models.
35         2. Values from the JSON file (if present and readable).
36         3. (Future) environment / CLI overrides.
37
38         Missing file or unreadable file ? returns a fully defaulted AppConfig.
39         Unknown keys in the file are tolerated (extra="allow" on the model).
40         """
41         cfg_path = Path(path) if path is not None else DEFAULT_CONFIG_PATH
42
43         file_data: dict[str, Any] = {}
44         if cfg_path.exists():
45             try:
46                 with open(cfg_path, "r", encoding="utf-8") as f:
47                     file_data = json.load(f)
48             except Exception as exc:
49                 # Non-fatal: continue with defaults + whatever we could parse.
50                 # In production we might log here.
51                 logger.warning(f"Failed to read {cfg_path}: {exc}. Using defaults + partial
data.")
52
53         # Merge: start from defaults, overlay file data (Pydantic will validate/coerce).
54         merged = {**_get_builtin_defaults(), **file_data}
55
56         return AppConfig.model_validate(merged)
57
58
59     def save_default_config(path: str | Path | None = None) -> Path:
60         """Write a fresh default config.json.
61
62         Used by setup.py diagnostics and the --setup CLI path.
63         Overwrites if the file exists.
64         """
65         cfg_path = Path(path) if path is not None else DEFAULT_CONFIG_PATH
66         cfg = AppConfig()
67         cfg_path.parent.mkdir(parents=True, exist_ok=True)

```

```

68         cfg_path.write_text(cfg.model_dump_json(indent=4), encoding="utf-8")
69         return cfg_path
70
71
72     # Backwards-compatible alias used by some older call sites during transition.
73     def get_default_config() -> dict[str, Any]:
74         """Return defaults as a plain dict (for transitional use only)."""
75         return _get_builtin_defaults()
76

```

8. user (2026-06-10T06:09:48.476Z)

docs\CODE_MAP.md:178:[Omitted long matching line]

docs\CODE_MAP.md:241:- **Gemini free-tier / API-key dead path:** hybrids + gemini return `[]` (not error) if `{PROVIDER}\_API\_KEY` missing OR WSL healthcheck fails ? a silent no-op. `analyze\_video` also returns `[]` on real failure -> empty != "no rallies". The reporting `silent\_provider\_noop`/`ai\_empty\_vs\_no\_rallies` rules exist precisely to disambiguate this in the report.

docs\BACKLOG.md:80:- ? **Item 3 (Error/Result contract) synergizes with the report.** Today the report INFERS failures (e.g. `silent\_provider\_noop` = 0 segments + no API key). A structured Result/error from providers/segmenters would let the report read the real reason as a first-class flag. **Revisit:** with item 3.

tests\test_reporting_harvest.py:77: def

test_silent_provider_noop_fires_when_no_key_and_zero_segments(monkeypatch):

tests\test_reporting_harvest.py:80: assert "silent_provider_noop" in _codes(env)

tests\test_reporting_harvest.py:82: flag = next(f for f in env.flags if f.code == "silent_provider_noop")

tests\test_reporting_harvest.py:91: assert "silent_provider_noop" not in codes

tests\test_reporting_harvest.py:102: assert "silent_provider_noop" not in codes

backend\reporting\harvest.py:140: present ? the signal the silent_provider_noop rule keys on. """

backend\reporting\spec.yaml:26: - code: silent_provider_noop

docs\archives\past_projects\low-regret-moves\LOW_REGRET_MOVES_PLAN.md:197:- **Item 3 (Error/Result contract) ? synergy.** A structured Result from providers/segmenters lets the report read failure reasons directly instead of inferring them (e.g. `silent\_provider\_noop`).

9. user (2026-06-10T06:10:11.694Z)

21: * `gemini` ? sends the entire video (or a `chunk\_duration` slice) to Google Gemini directly. Highest precision per call, highest cost.

23: * **Pluggable AI Provider Registry** (`backend/providers/`): The Gemini provider ships in-tree; new providers (e.g. an OpenAI or local-model backend) can be registered without touching the segmenters.

79: python -m backend.main --video-path "C:/path/to/my_video.mp4" --segmenter gemini

109: "ai_provider": "gemini",

110: "ai_model": "gemini-2.5-flash",

155: * `indexing.default\_segmenter` ? `yolo\_hybrid` | `gemini` | `motion`.

197:### Q7: How do I configure and run an AI-powered segmenter (Gemini / YOLO-hybrid)?

198:Both `yolo\_hybrid` (default) and `gemini` need an API key for the AI provider. To use Google Gemini:

202: GEMINI_API_KEY=your_key_here

206: # Default: cheap candidate filtering with YOLO, then precise boundaries via Gemini

209: # Full-video Gemini analysis (more expensive, highest semantic precision per call)

210: python -m backend.main --video-path "C:/path/to/video.mp4" --segmenter gemini

215: *Or set `default\_segmenter`: "yolo_hybrid" (or `gemini`/"motion") under `indexing` in `config.json`.*

217:The AI provider used by `yolo\_hybrid` and `gemini` is controlled by `ai\_provider` + `ai\_model` at the top level of `config.json`. Today only `gemini` is registered; new providers can be added under `backend/providers/`.

219:[Omitted long matching line]

227:4. **DB population** ? confirmed segments are written to `play\_segments` along with `shot\_count`, `shot\_count\_confidence`, and a `detector` tag like `yolo-hybrid-gemini-2.5-flash`.

229:In practice this typically cuts AI-provider runtime/cost by an order of magnitude versus uploading the full video to `gemini` directly, while preserving the LLM's strength at semantic boundary detection.

```
231:### Q9: How can I debug/test the Gemini Segmenter on a small portion of a video without
uploading a huge multi-GB file?
237:     "default_segmenter": "gemini",
242:3. When `"debug_duration"` is set, the Gemini segmenter will instantly run a zero-re-encode,
sub-second `ffmpeg` slice to create a tiny temporary clip of the first 15 seconds, upload only
that tiny clip to Google AI Studio (which takes <1 second to upload and <5 seconds to process),
run the model query, and clean up the temporary files automatically. This allows you to iterate
on model prompts, evaluate validation overlaps, and verify rally boundaries in under 10 seconds!
245:[Omitted long matching line]
309:The suite mocks external dependencies (the Gemini client, ffprobe/ffmpeg, OpenCV
`VideoCapture`, the YOLO model) so it runs offline and without a GPU. The tests cover:
315:* `tests/test_providers.py` + `tests/test_provider_gemini.py` ? AI provider registry and
the Gemini provider
317:* `tests/test_gemini.py` ? `GeminiPlaySegmenter` (single-file and chunked paths)
331:  indexer/                # Pluggable rally segmenters: motion, gemini, yolo_hybrid,
tracknet_hybrid
333:  providers/              # Pluggable AI providers (gemini today)
```

10. user (2026-06-10T06:10:51.653Z)

Structured output provided successfully

11. assistant (2026-06-10T06:10:54.927Z)

Verification complete.